

GymTrackPro v2.0.0 RC1

Project Specification & Build Guide

Cross-Platform Gym Membership Management System

Prepared By

Project Team (3 Members)

Leader: _____

Member: _____

Member: _____

Version

Version 2.0.0 RC1

Document Purpose

This document serves as the official development guide for GymTrackPro. It contains the project's requirements, architecture, design decisions, modules, database structure, development standards, and implementation roadmap.

This document should be followed by all members of the development team throughout the project lifecycle.

It also serves as the primary reference for the Antigravity IDE Agent when generating code, ensuring consistency and reducing unnecessary assumptions during development.

Table of Contents

Chapter 1 – Introduction

Chapter 2 – Agent Instructions

Chapter 3 – Development Strategy

Chapter 4 – Team Collaboration

Chapter 5 – GitHub Workflow

Chapter 6 – System Architecture

Chapter 7 – Offline-First Architecture

Chapter 8 – Security

Chapter 9 – User Roles

Chapter 10 – System Modules

Chapter 11 – Database Design

Chapter 12 – Business Rules

Chapter 13 – API Overview

Chapter 14 – Application Navigation

Chapter 15 – Folder Structure

Chapter 16 – Development Standards

Chapter 17 – Standard Error Messages

Chapter 18 – Development Roadmap

Chapter 19 – Future Improvements

Conclusion

Chapter 1

Introduction

Project Overview

GymTrackPro is a cross-platform Gym Membership Management System developed to replace traditional paper-based record keeping with a centralized digital solution.

Many small and medium-sized gyms still rely on manual attendance sheets, handwritten payment records, and physical membership logs. These methods are inefficient, prone to errors, and difficult to maintain as the number of members grows.

GymTrackPro addresses these challenges by providing a modern application that allows gym staff to manage members, memberships, attendance, payments, reports, and system settings in a single platform.

The application is designed using an Offline-First approach, ensuring that important operations can continue even during temporary internet outages. Once connectivity is restored, locally stored data is synchronized with the online database.

Project Objectives

The project aims to:

- Digitize gym management processes.
 - Reduce manual paperwork.
 - Improve member registration.
 - Track attendance efficiently.
 - Record membership payments accurately.
 - Monitor membership status.
 - Generate useful reports.
 - Secure sensitive information through authentication and authorization.
 - Allow offline operations using SQLite.
 - Synchronize data with the online database automatically.
-

Project Scope

GymTrackPro includes the following modules:

- Authentication
- Dashboard
- User Management
- Member Management
- Membership Plans
- Membership Subscriptions
- Membership Pause
- Payments
- Attendance
- QR Attendance
- Digital Membership Card
- Walk-In Visitors
- Notifications
- Reports
- Settings
- Audit Logs

The following features are outside the scope of Version 2.0:

- AI Assistant
 - AI Chatbot
 - Workout Recommendation
 - Nutrition Recommendation
 - Trainer Management
 - Equipment Inventory
 - Payroll System
 - Multi-Branch Support
-

Target Users

Administrator

Responsible for managing the entire system, including members, users, settings, reports, and overall gym operations.

Receptionist

Responsible for member registration, attendance, payment recording, and daily front-desk operations.

Gym Members

Gym members primarily interact with the system through QR code attendance and their digital membership card.

Expected Benefits

For Gym Owners

- Centralized member information
 - Accurate payment records
 - Easier report generation
 - Improved attendance monitoring
-

For Receptionists

- Faster registration process
 - Faster attendance recording
 - Easier payment management
 - Simplified member lookup
-

For Members

- Faster check-in
 - Digital membership identification
 - More accurate membership records
-

Project Vision

The vision of GymTrackPro is to provide a reliable, user-friendly, and modern gym membership management system that simplifies daily operations while remaining easy to maintain and expand.

The project focuses on clean software architecture, maintainable code, and practical features suitable for small to medium-sized fitness centers.

Chapter 2

Agent Instructions

Purpose

The Antigravity IDE Agent will use this document as its primary reference during software development.

The agent should follow the architecture, business rules, and development standards described in this document without introducing unnecessary complexity or undocumented features.

General Rules

The IDE Agent should:

- Follow the architecture described in this document.
 - Generate clean and maintainable code.
 - Use MVVM architecture.
 - Use the Repository Pattern.
 - Use Dependency Injection.
 - Follow REST API principles.
 - Keep modules independent whenever possible.
 - Reuse existing code before creating new implementations.
 - Complete one module before moving to the next.
-

The Agent Must Not

- Invent undocumented modules.
- Change the database structure without approval.
- Introduce unnecessary dependencies.
- Mix UI logic with business logic.
- Skip input validation.
- Duplicate code.
- Ignore business rules.
- Remove offline functionality.
- Replace the project architecture.

Development Philosophy

GymTrackPro is a student capstone project.

The goal is to build a clean, complete, and maintainable application—not an enterprise-scale platform.

Development should prioritize:

- Simplicity
 - Readability
 - Maintainability
 - Consistency
 - Practical implementation
-

Chapter 3

Development Strategy

GymTrackPro is developed using an incremental approach where each module is completed individually before moving to the next.

Each completed module should include:

- Database implementation
- Repository implementation
- Service implementation
- API implementation
- ViewModel
- User Interface
- Validation
- Testing

before proceeding to another module.

Development Methodology

The team will follow Agile principles.

Development will occur in small iterations, allowing testing and improvements throughout the project.

Team Size

Three Developers

Development Priorities

Priority 1

Core functionality

Priority 2

Stable database

Priority 3

Offline synchronization

Priority 4

User Interface improvements

Priority 5

Performance optimization

Code Quality Goals

The project should maintain:

- Modular code
 - Reusable components
 - Consistent naming
 - Small methods
 - Easy maintenance
-

Chapter 4

Team Collaboration

GymTrackPro is developed by a team of three student developers.

Although responsibilities are divided among team members, everyone is responsible for maintaining code quality and ensuring the project follows this specification.

Team Roles

Lead Developer

Responsible for:

- Project architecture
 - Backend API
 - Database
 - Authentication
 - Final integration
 - Code reviews
-

Frontend Developer

Responsible for:

- User Interface
 - Dashboard
 - Member Management
 - Reports
 - Settings
-

Backend & QA Developer

Responsible for:

- Attendance
- Payments
- QR Attendance
- Testing
- Bug Fixes
- Documentation

Collaboration Guidelines

Every developer should:

- Pull the latest changes before starting work.
 - Use feature branches.
 - Test features before merging.
 - Write meaningful commit messages.
 - Resolve merge conflicts immediately.
 - Avoid modifying another developer's module without discussion.
-

Project Rules

- One feature per branch.
 - One pull request per completed feature.
 - Merge only after testing.
 - Follow the architecture in this document.
 - Keep commits focused and descriptive.
 - Do not bypass code reviews for major changes.
-

Chapter 5

GitHub Workflow

Repository Structure

main

develop

feature/*

hotfix/*

Branch Naming

Examples

feature/login

feature/dashboard

feature/members

feature/payments

feature/attendance

feature/reports

feature/settings

feature/users

Development Workflow

Create Feature Branch

↓

Develop Feature



Local Testing



Commit Changes



Push to GitHub



Create Pull Request



Team Review



Merge into develop



Final Testing

↓

Merge into main

Commit Message Examples

Add member registration

Implement JWT authentication

Fix QR attendance validation

Improve payment history page

Update membership renewal logic

Merge Guidelines

Before merging, ensure:

- The feature works correctly.
 - No existing functionality is broken.
 - The code follows project standards.
 - Validation has been implemented.
 - The feature has been tested locally.
-

End of Part 1

This is now significantly stronger than the original PDF while still remaining appropriate for a student capstone project.

Part 2 will continue with:

- Chapter 6 – System Architecture
- Chapter 7 – Offline-First Architecture
- Chapter 8 – Security
- Chapter 9 – User Roles

These chapters will incorporate all the refinements we've agreed on, including offline synchronization, database sync rules, expanded business rules, and the User Management module.

Perfect. Let's continue with the **actual specification**.

Chapter 6

System Architecture

Overview

GymTrackPro follows a **Client–Server Architecture** using a modern layered design. The mobile application communicates with a secure REST API, which handles all business logic and database operations.

The application is also designed with an **Offline-First** approach. When an internet connection is unavailable, the system continues to operate using a local SQLite database. Once connectivity is restored, pending changes are synchronized with the online MySQL database.

This architecture provides reliability, maintainability, and scalability while remaining suitable for a student capstone project.

Overall Architecture

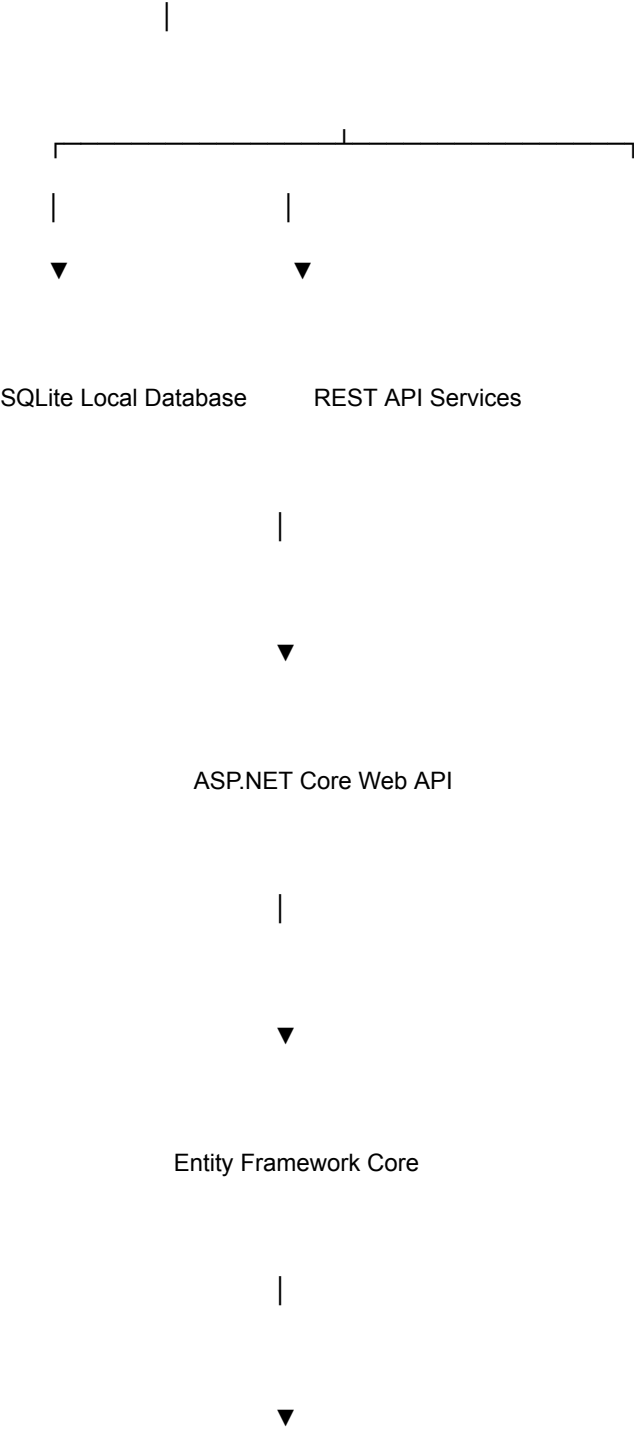
GymTrackPro Mobile App

(.NET MAUI)

|



MVVM Architecture



Architecture Layers

Presentation Layer

Responsible for displaying the application's user interface.

Components include:

- Views
- XAML Pages
- User Controls

Examples

- LoginPage
 - DashboardPage
 - MembersPage
 - PaymentsPage
-

ViewModel Layer

Responsible for handling user interactions and communicating with services.

Examples

- LoginViewModel
- DashboardViewModel
- MembersViewModel
- PaymentsViewModel

The ViewModel should never communicate directly with the database.

Service Layer

Responsible for business logic.

Examples

- AuthenticationService
- MemberService
- PaymentService
- AttendanceService
- NotificationService

Responsibilities include:

- Data validation
 - Business rules
 - API communication
 - Synchronization requests
-

Repository Layer

Responsible for database access.

Repositories isolate database operations from the rest of the application.

Examples

- UserRepository
- MemberRepository
- PaymentRepository
- AttendanceRepository

This allows database changes without affecting business logic.

Database Layer

GymTrackPro uses two databases:

SQLite

- Local storage
- Offline support
- Pending synchronization

MySQL

- Main online database
 - Permanent storage
 - Shared across devices
-

Why MVVM?

The project follows the MVVM (Model-View-ViewModel) pattern because it:

- Separates UI from business logic
- Makes code easier to maintain
- Improves code organization
- Supports dependency injection

- Simplifies testing
 - Encourages reusable components
-

Repository Pattern

The Repository Pattern provides a clean interface between business logic and the database.

Benefits include:

- Easier maintenance
 - Cleaner code
 - Database independence
 - Improved testing
 - Reduced duplication
-

Dependency Injection

Dependency Injection (DI) is used throughout the project to manage services and repositories.

Benefits include:

- Loose coupling
 - Easier testing
 - Better maintainability
 - Simplified object creation
-

REST API

The mobile application never communicates directly with the MySQL database.

Instead, all online requests pass through the ASP.NET Core Web API.

This provides:

- Authentication
 - Authorization
 - Validation
 - Business logic
 - Secure database access
-

Chapter 7

Offline-First Architecture

Purpose

GymTrackPro is designed to continue operating even when internet connectivity is unavailable.

Many gyms experience temporary internet outages. Instead of preventing users from working, GymTrackPro stores data locally and synchronizes it later.

Local Database

SQLite serves as the local database.

Data stored locally includes:

- Members
- Membership Plans
- Memberships
- Attendance
- Payments
- Notifications
- Settings

This allows daily operations to continue without interruption.

Online Database

MySQL serves as the primary online database.

All synchronized records are permanently stored here.

Synchronization Queue

Whenever a user creates or updates data while offline:

1. Save to SQLite
2. Add to Pending Sync Queue
3. Wait for internet

4. Automatically upload changes

This prevents data loss.

Network Detection

The application continuously monitors internet connectivity.

When a connection becomes available:

- Upload pending records
- Download updated records
- Refresh local database
- Mark synchronized records

This process should happen automatically whenever possible.

Synchronization Flow

User Creates Record

↓

Save to SQLite

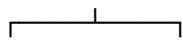
↓

Pending Sync Queue

↓

Internet Available?

|



NO YES



Wait Upload to API



MySQL Database



Mark Record Synced

Database Synchronization Rules

New Records

- Save immediately to SQLite.
- Mark as pending synchronization.

- Upload automatically once connected.
-

Updated Records

- Update local copy.
 - Add to synchronization queue.
 - Upload latest changes.
-

Deleted Records

Records should not be permanently removed immediately.

Instead:

- Mark as deleted.
 - Synchronize deletion.
 - Remove locally only after successful synchronization.
-

Conflict Resolution

If two devices modify the same record before synchronization:

Compare Last Modified Timestamp

↓

Newest Update Wins

↓

Create Audit Log

↓

Notify Administrator

This simple approach is suitable for the project's scope.

Synchronization Failure

If synchronization fails:

- Keep the local copy.
 - Retry automatically later.
 - Display synchronization status.
 - Prevent data loss.
-

Offline Features

The following features should work without internet:

- Login (previously authenticated user)
- View Members
- Register Members
- Attendance
- Payments
- Notifications
- Membership Information

Reports can be generated using available local data, though they may not reflect changes made on other devices until synchronization is complete.

Chapter 8

Security

Purpose

GymTrackPro protects user accounts, member information, attendance records, and payment data through secure authentication and authorization.

Authentication

Users must log in before accessing the system.

Authentication includes:

- Username
 - Password
 - JWT Token
-

Password Security

Passwords must never be stored as plain text.

The system stores only hashed passwords.

Benefits:

- Better security
 - Protection against database leaks
 - Compliance with good development practices
-

Role-Based Authorization

GymTrackPro supports two system roles.

Administrator

Full system access.

Receptionist

Limited access based on assigned permissions.

Secure Communication

When deployed online, all communication between the mobile application and the API should use HTTPS.

Input Validation

Every form must validate user input before saving.

Examples:

- Required fields
 - Valid email format
 - Valid phone number
 - Positive payment amount
 - Existing member validation
 - Membership status validation
-

Secure Token Storage

Authentication tokens should be stored securely on the device and removed when the user logs out or the session expires.

Audit Logging

The following actions should automatically generate audit logs:

- Login
 - Logout
 - Add Member
 - Edit Member
 - Record Payment
 - Renew Membership
 - Pause Membership
 - Resume Membership
 - Create User
 - Disable User
 - Update Settings
-

Session Management

The application should:

- Remember active sessions securely
 - Allow manual logout
 - Expire invalid sessions
 - Require login after expiration
-

Chapter 9

User Roles

GymTrackPro currently supports two primary user roles.

Administrator

The Administrator has full control over the system.

Permissions include:

- Dashboard
- User Management
- Member Management
- Membership Plans
- Memberships
- Membership Pause
- Payments
- Attendance
- QR Attendance
- Walk-In Visitors
- Reports
- Notifications
- Settings
- Audit Logs

Administrators are also responsible for creating and managing system users.

Receptionist

Receptionists handle day-to-day gym operations.

Responsibilities include:

- Register Members
- Update Member Information
- Record Payments
- Record Attendance
- Scan QR Codes
- View Reports
- View Notifications

Receptionists cannot:

- Create users
 - Delete users
 - Modify system settings
 - View audit logs
 - Change system configurations
-

User Management Module (Admin Only)

Purpose

Allow administrators to manage accounts used to access the system.

Features

- Create User
 - Edit User
 - Disable User
 - Enable User
 - Reset Password
 - Change Password
 - Assign Roles
 - View User List
-

Business Rules

- Only Administrators can access User Management.
 - Usernames must be unique.
 - Passwords must always be hashed.
 - Disabled accounts cannot log in.
 - Every user action should be recorded in the Audit Log.
-

Permission Matrix

Module	Administrator	Receptionist
--------	---------------	--------------

Dashboard		
User Management		
Member Management		
Membership Plans		View Only
Memberships		
Membership Pause		
Payments		
Attendance		
QR Attendance		
Walk-In Visitors		
Reports		
Notifications		View Only
Settings		

Audit Logs



End of Part 2

At this point, the document has moved beyond a simple project proposal—it now defines the **architecture, offline behavior, security model, and user permissions** that will guide development.

In **Part 3**, we'll cover the heart of the project: **all system modules** (Authentication, Dashboard, Member Management, Memberships, Payments, Attendance, QR Attendance, Reports, Settings, Audit Logs, and more), rewritten into clear implementation specifications that the Antigravity IDE Agent can scaffold directly. This will be the largest section of the document.

Perfect. This is the biggest chapter of the document. Instead of simply listing features, we're defining exactly **how each module should behave** so the Antigravity IDE Agent has a clear implementation guide.

Chapter 10

System Modules

This chapter describes every module included in GymTrackPro. Each module includes its purpose, features, business rules, validation requirements, and expected pages.

Module 1 — Authentication

Purpose

The Authentication module controls access to the application by verifying user credentials and assigning permissions based on user roles.

Features

- User Login
 - User Logout
 - Remember Session
 - Role-Based Access
 - Secure Authentication
 - Session Expiration
-

Inputs

- Username
 - Password
-

Outputs

- JWT Token
 - User Information
 - Assigned Role
-

Validation

- Username is required.
 - Password is required.
 - Invalid credentials should display an error message.
 - Disabled accounts cannot log in.
-

Business Rules

- Only registered users may access the system.
 - Passwords must be stored as hashes.
 - Users must have an assigned role.
 - Sessions expire automatically after a configurable period of inactivity.
-

Pages

- LoginPage

Module 2 — Dashboard

Purpose

Provide administrators and receptionists with a quick overview of gym operations.

Dashboard Widgets

Membership Summary

- Total Members
 - Active Members
 - Expired Members
 - Paused Members
-

Attendance Summary

- Today's Attendance
 - Monthly Attendance
 - Current Members Inside (Live Counter)
-

Revenue Summary

- Daily Revenue
 - Monthly Revenue
-

Recent Activities

Display recent:

- Member Registrations
 - Membership Renewals
 - Attendance Logs
 - Payments
-

Upcoming Membership Expirations

Display members whose memberships will expire within:

- 7 Days
 - 15 Days
 - 30 Days
-

Quick Actions

Buttons for:

- Register Member
 - Record Payment
 - Record Attendance
 - Scan QR
 - View Reports
-

Business Rules

Dashboard statistics should update automatically whenever new records are added or synchronized.

Pages

- DashboardPage
-

Module 3 — User Management (Administrator Only)

Purpose

Manage accounts that can access the system.

Features

- View Users
 - Create User
 - Edit User
 - Disable User
 - Enable User
 - Reset Password
 - Change Password
 - Assign Role
-

User Information

- User ID
 - Username
 - Full Name
 - Role
 - Account Status
 - Date Created
 - Last Login
-

Validation

- Username must be unique.
 - Password must meet minimum requirements.
 - Role selection is required.
-

Business Rules

- Only Administrators can access this module.
 - Disabled accounts cannot log in.
 - Every action must be recorded in Audit Logs.
-

Pages

- UsersPage
 - AddUserPage
 - EditUserPage
-

Module 4 — Member Management

Purpose

Store and manage all gym member information.

Features

Member Registration

Register new members.

Member Details

View complete member information.

Edit Member

Update member details.

Deactivate Member

Deactivate members instead of permanently deleting records.

Search

Search by:

- Member ID
 - Name
 - Phone Number
 - QR Code
-

Filter

Filter members by:

- Active
 - Expired
 - Paused
 - Inactive
-

Sort

Sort by:

- Name
 - Registration Date
 - Expiration Date
-

Pagination

Display member records in manageable pages.

Profile Picture

Upload or update a member profile picture.

Membership Badge

Display current membership status using colored badges.

Example:

-  Active
 -  Paused
 -  Expired
-

Member Information

Personal Information

- Member ID
- First Name
- Last Name
- Gender
- Date of Birth

Contact Information

- Phone Number
- Email Address
- Address
- Emergency Contact

Membership Information

- Registration Date
- Membership Status
- Assigned Plan
- Expiration Date

Additional Information

- QR Code
- Profile Picture

Validation

- First Name required.
- Last Name required.
- Phone Number required.
- Emergency Contact required.
- QR Code must be unique.
- Phone Number must be unique.
- Email should be unique if provided.

Business Rules

- Member IDs are generated automatically.
- Members cannot be permanently deleted.
- Inactive members remain in reports for historical purposes.

Pages

- MembersPage
 - AddMemberPage
 - EditMemberPage
 - MemberDetailsPage
-

Module 5 — Membership Plans

Purpose

Manage available membership plans offered by the gym.

Features

- Add Plan
 - Edit Plan
 - View Plans
 - Deactivate Plan
-

Plan Information

- Plan ID
 - Plan Name
 - Duration (Days)
 - Price
 - Description
 - Status
-

Validation

- Plan Name required.
 - Price must be greater than zero.
 - Duration must be greater than zero.
-

Business Rules

- Plans currently assigned to members cannot be deleted.

- Deactivated plans cannot be assigned to new members.
-

Pages

- MembershipPlansPage
 - AddPlanPage
 - EditPlanPage
-

Module 6 — Membership Subscriptions

Purpose

Assign and manage member subscriptions.

Features

- Assign Membership Plan
 - Renew Membership
 - View Subscription History
 - View Membership Status
-

Subscription Information

- Subscription ID
 - Member
 - Plan
 - Start Date
 - End Date
 - Status
-

Membership Status

- Active
- Expired
- Paused

Business Rules

- One active subscription per member.
 - Renewing extends the expiration date.
 - Expired memberships cannot check in.
 - Paused memberships cannot check in.
-

Pages

- MembershipPage
 - RenewMembershipPage
-

Module 7 — Membership Pause

Purpose

Temporarily suspend a member's subscription.

Features

- Pause Membership
 - Resume Membership
 - Pause History
-

Pause Information

- Pause ID
 - Subscription
 - Start Date
 - End Date
 - Reason
-

Business Rules

- Membership expiration should be adjusted based on the pause duration.
 - Members cannot check in while paused.
-

Pages

- PauseMembershipPage
-

Module 8 — Payments

Purpose

Record and monitor membership payments.

Features

- Record Payment
 - Payment History
 - Payment Status
 - Printable Receipt
 - Receipt History
-

Payment Methods

- Cash
 - GCash
 - Card
 - Bank Transfer
-

Payment Status

- Paid
 - Partial
 - Pending
-

Payment Information

- Payment ID
 - Member
 - Subscription
 - Amount
 - Discount
 - Final Amount
 - Payment Method
 - Payment Status
 - Receipt Number
 - Reference Number
 - Date Paid
-

Validation

- Amount must be greater than zero.
 - Member must exist.
 - Payment method required.
-

Business Rules

- Every payment generates a receipt.
 - Payment cannot exceed the remaining balance.
 - Payment records should use soft delete if removal is required.
-

Pages

- PaymentsPage
 - AddPaymentPage
 - PaymentHistoryPage
 - ReceiptPage
-

Module 9 — Attendance

Purpose

Track member attendance and gym usage.

Features

- Check-In
 - Check-Out
 - Attendance History
 - Attendance Calendar
 - Live Attendance Counter
-

Attendance Information

- Attendance ID
 - Member
 - Check-In Time
 - Check-Out Time
 - Attendance Date
-

Validation

- Member must have an active membership.
 - Duplicate attendance is not allowed.
-

Business Rules

- One check-in per member per day.
 - Check-out cannot occur before check-in.
 - Expired memberships cannot check in.
 - Paused memberships cannot check in.
-

Pages

- AttendancePage
 - AttendanceHistoryPage
-

Module 10 — QR Attendance

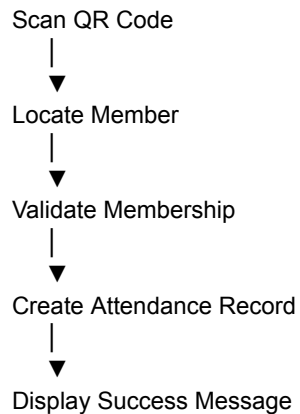
Purpose

Provide a faster and more convenient way to record attendance.

Features

- Generate QR Code
 - Scan QR Code
 - Validate Membership
 - Automatic Check-In
-

QR Attendance Flow



Business Rules

- QR Codes must be unique.
 - Invalid QR Codes should display an error.
 - Expired or paused memberships should be rejected.
-

Pages

- QRScannerPage
-

Module 11 — Digital Membership Card

Purpose

Provide members with a digital identification card.

Card Information

- Profile Picture
 - Member Name
 - Member ID
 - Membership Plan
 - Membership Status
 - Expiration Date
 - QR Code
-

Features

- View Digital Card
 - Refresh QR Code (if needed)
 - Display Membership Status
-

Pages

- DigitalCardPage
-

Module 12 — Walk-In Visitors

Purpose

Record visitors who are not registered members.

Features

- Register Visitor
- Record Walk-In Fee
- View Visitor History

Visitor Information

- Visitor ID
- Full Name
- Visit Date
- Fee Paid
- Purpose

Business Rules

- Walk-in visitors cannot use member QR codes.
- Walk-in records should remain available for reporting.

Pages

- WalkInVisitorsPage

Module 13 — Notifications

Purpose

Provide important reminders and alerts to users.

Notification Types

- Membership Expiring
- Membership Expired
- Payment Reminder
- Renewal Reminder
- Synchronization Status

Features

- View Notifications
 - Mark as Read
 - Automatic Notification Generation
-

Business Rules

- Notifications should be generated automatically based on system events.
 - Read notifications remain available until archived.
-

Pages

- NotificationsPage
-

End of Part 3A

The next section (**Part 3B**) will complete the remaining modules:

- Reports
- Settings
- Audit Logs

Then we'll move into:

- Database Design (with PK/FK/Indexes)
- Expanded Business Rules
- API Overview
- UI Navigation
- Folder Structure
- Development Standards
- Error Messages
- Development Roadmap
- Future Improvements
- Conclusion

After that, the specification will be complete and ready to turn into your PDF.

next

Perfect. We're now finishing the specification. After this part, your document will be essentially complete.

Module 14 — Reports

Purpose

The Reports module provides administrators and authorized users with summarized information about gym operations, member activity, attendance, and financial transactions. Reports help monitor performance, identify trends, and support better decision-making.

Features

Revenue Reports

Generate revenue summaries based on:

- Daily Revenue
 - Weekly Revenue
 - Monthly Revenue
 - Custom Date Range
-

Attendance Reports

Generate reports showing:

- Daily Attendance
 - Weekly Attendance
 - Monthly Attendance
 - Attendance by Member
-

Membership Reports

Generate reports for:

- Active Members
 - Expired Members
 - Paused Memberships
 - Newly Registered Members
 - Renewed Memberships
-

Payment Reports

Generate reports showing:

- Paid Payments
 - Partial Payments
 - Pending Payments
 - Total Revenue
-

Export Features

Reports can be exported as:

- PDF
 - Excel
-

Report Filters

Users can filter reports by:

- Date Range
 - Membership Plan
 - Member
 - Payment Status
-

Business Rules

- Reports should only display data the user is authorized to access.
 - Exported reports should include the generation date and time.
 - Reports should retrieve synchronized data whenever internet is available.
-

Pages

- ReportsPage
 - RevenueReportPage
 - AttendanceReportPage
 - MembershipReportPage
 - PaymentReportPage
-

Module 15 — Settings

Purpose

The Settings module allows administrators to configure application preferences and gym information.

Gym Information

Administrators can manage:

- Gym Name
 - Address
 - Contact Number
 - Business Hours
 - Gym Logo
-

Membership Settings

Configure:

- Default Membership Duration
 - Renewal Reminder Days
 - Membership Status Colors
-

Discount Settings

Configure available discounts such as:

- Student Discount
 - Senior Citizen Discount
 - Promotional Discount
-

Application Preferences

- Light Theme
 - Dark Theme
-

Business Rules

- Only Administrators may modify settings.
 - Receptionists may view selected information but cannot edit settings.
 - Changes should be recorded in the Audit Log.
-

Pages

- SettingsPage
-

Module 16 — Audit Logs

Purpose

Track important activities performed within the system for accountability and troubleshooting.

Logged Activities

The system records actions such as:

- User Login
 - User Logout
 - Create User
 - Update User
 - Disable User
 - Register Member
 - Edit Member
 - Renew Membership
 - Pause Membership
 - Resume Membership
 - Record Payment
 - Generate Report
 - Update Settings
 - Synchronization Events
-

Audit Log Information

Each log should include:

- Log ID
- User

- Action
 - Module
 - Description
 - Date
 - Time
-

Features

- View Logs
 - Search Logs
 - Filter by User
 - Filter by Date
 - Filter by Module
-

Business Rules

- Audit logs cannot be edited.
 - Audit logs cannot be deleted through the application.
 - Only Administrators can access Audit Logs.
-

Pages

- AuditLogsPage
-

Chapter 11

Database Design

Overview

GymTrackPro uses a dual-database approach.

- **SQLite** is used for offline storage on the mobile device.
- **MySQL** is used as the central online database.

The database design emphasizes data integrity, normalization, and maintainability.

Users Table

Purpose

Stores system user accounts.

Columns

UserID (PK)

Username

PasswordHash

FullName

Role

IsActive

CreatedAt

LastLogin

Indexes

Primary Key

UserID

Unique

Username

Members Table

Purpose

Stores member information.

Columns

MemberID (PK)

FirstName

LastName

Gender

BirthDate

PhoneNumber

Email

Address

EmergencyContact

ProfilePicture

QRCode

Status

DateRegistered

Indexes

Primary Key

MemberID

Unique

PhoneNumber

Email

QRCode

MembershipPlans Table

Columns

PlanID (PK)

PlanName

DurationDays

Price

Description

Status

Indexes

Primary Key

PlanID

Unique

PlanName

Subscriptions Table

Columns

SubscriptionID (PK)

MemberID (FK)

PlanID (FK)

StartDate

EndDate

Status

MembershipPause Table

Columns

PauseID (PK)

SubscriptionID (FK)

StartDate

EndDate

Reason

Payments Table

Columns

PaymentID (PK)

MemberID (FK)

SubscriptionID (FK)

Amount

Discount

FinalAmount

PaymentMethod

PaymentStatus

ReceiptNumber

ReferenceNumber

DatePaid

Attendance Table

Columns

AttendanceID (PK)

MemberID (FK)

AttendanceDate

CheckInTime

CheckOutTime

WalkInVisitors Table

Columns

VisitorID (PK)

VisitorName

VisitDate

FeePaid

Purpose

Notifications Table

Columns

NotificationID (PK)

Title

Message

Status

CreatedAt

AuditLogs Table

Columns

LogID (PK)

UserID (FK)

Action

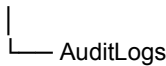
Module

Description

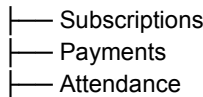
Timestamp

Database Relationships

Users



Members



MembershipPlans
└─ Subscriptions

Subscriptions
└─ Payments
└─ MembershipPause

Chapter 12

System Business Rules

Member Rules

- Every member must have a unique Member ID.
 - Phone numbers must be unique.
 - Email addresses should be unique if provided.
 - QR Codes must be unique.
 - Members cannot have more than one active subscription.
 - Inactive members cannot renew memberships.
 - Expired members cannot check in.
 - Paused members cannot check in.
-

Membership Rules

- Only one active membership is allowed.
 - Membership renewal extends the expiration date.
 - Deactivated plans cannot be assigned.
 - Membership status updates automatically when expired.
-

Payment Rules

- Payment amount must be greater than zero.
 - Payments cannot exceed the remaining balance.
 - Every successful payment generates a receipt.
 - Deleted payment records should use soft delete.
 - Payments must belong to an existing member.
-

Attendance Rules

- One check-in per day.
 - Check-out cannot happen before check-in.
 - Walk-in visitors cannot use member QR codes.
 - Attendance requires an active membership.
-

User Rules

- Only Administrators can manage users.
 - Usernames must be unique.
 - Disabled users cannot log in.
 - Every user action should be logged.
-

Walk-In Visitor Rules

- Walk-in visitors are not members.
 - Walk-in visitors cannot renew memberships without first becoming registered members.
-

Chapter 13

API Overview

GymTrackPro communicates through a RESTful API.

Authentication

POST /api/auth/login

POST /api/auth/logout

Users

GET /api/users

POST /api/users

PUT /api/users/{id}

PATCH /api/users/{id}/disable

Members

GET /api/members

GET /api/members/{id}

POST /api/members

PUT /api/members/{id}

Membership Plans

GET /api/plans

POST /api/plans

PUT /api/plans/{id}

Memberships

GET /api/subscriptions

POST /api/subscriptions

POST /api/subscriptions/{id}/renew

POST /api/subscriptions/{id}/pause

POST /api/subscriptions/{id}/resume

Payments

GET /api/payments

POST /api/payments

Attendance

POST /api/attendance/checkin

POST /api/attendance/checkout

POST /api/attendance/qr

Reports

GET /api/reports/revenue

GET /api/reports/attendance

GET /api/reports/membership

End of Part 3B

The **final part** will wrap up the specification with:

- Application Navigation
- Folder Structure
- Development Standards
- Database Synchronization Rules
- Standard Error Messages
- Development Roadmap (Build Order)
- Future Improvements
- Conclusion

That will complete the **GymTrackPro v2.0.0 RC1 Project Specification & Build Guide**.

next

Perfect. This is the **final chapter** of the specification. After this, I'd consider the architecture **frozen** and recommend starting development instead of continuing to add features.

Chapter 14

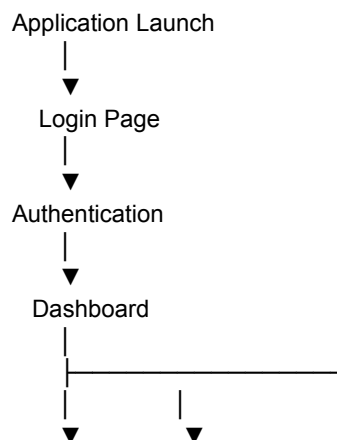
Application Navigation

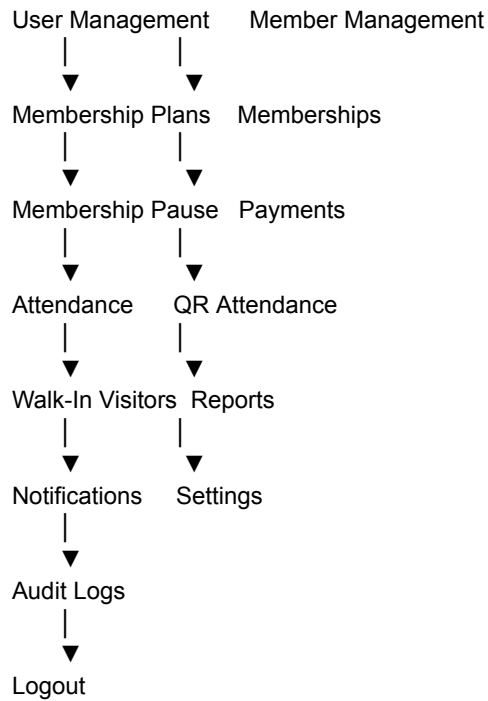
Overview

The application follows a simple and consistent navigation flow to make daily operations easy for both administrators and receptionists.

Users are directed to the Dashboard after successful authentication. From there, they can access the modules available to their assigned role.

Navigation Flow





Navigation Principles

The application should:

- Keep navigation simple.
- Minimize unnecessary page transitions.
- Display only modules available to the logged-in user's role.
- Return users to the Dashboard after completing major operations when appropriate.

Chapter 15

Folder Structure

The project is divided into two main solutions:

- Mobile Application
- Web API

Mobile Project

GymTrackPro.Mobile

- |
 - |— Views
 - |— ViewModels
 - |— Models
 - |— Services
 - |— Repositories
 - |— DTOs
 - |— Helpers
 - |— Resources
 - |— Controls
 - |— Converters
 - |— Validators
 - |— Data
 - |— Sync
 - |— Interfaces
 - |— AppShell.xaml
-

API Project

GymTrackPro.API

- |
- |— Controllers
- |— Services
- |— Repositories
- |— Interfaces
- |— Models
- |— DTOs
- |— Middleware

|— Authentication

|— Validators

|— Helpers

|— Data

|— Configurations

|— Migrations

|— Program.cs

Why This Structure?

This organization keeps responsibilities separated, making the project easier to understand, maintain, and expand.

Each folder has a clear purpose, reducing code duplication and improving collaboration among team members.

Chapter 16

Development Standards

Coding Standards

All developers should follow consistent coding practices throughout the project.

Guidelines include:

- Use meaningful variable and method names.
 - Keep methods focused on a single responsibility.
 - Avoid duplicate code.
 - Separate UI from business logic.
 - Keep files organized according to the folder structure.
 - Comment only when necessary to explain complex logic.
-

Architecture Standards

The project must consistently follow:

- MVVM Pattern
- Repository Pattern
- Dependency Injection
- REST API Architecture

No module should bypass these architectural principles.

Git Standards

- One feature per branch.
 - One pull request per completed feature.
 - Write clear commit messages.
 - Test before merging.
 - Resolve merge conflicts promptly.
 - Keep the `main` branch stable.
-

Code Review Checklist

Before approving a pull request, verify:

- Feature works correctly.
 - No unnecessary code was added.
 - Business rules are followed.
 - Validation is complete.
 - UI is consistent with the rest of the application.
 - No existing functionality was broken.
-

Chapter 17

Database Synchronization Rules

Overview

GymTrackPro supports both offline and online operation.

SQLite stores local data, while MySQL stores synchronized records.

New Records

When a user creates a record:

1. Save to SQLite.
 2. Add to the synchronization queue.
 3. Upload to MySQL when internet becomes available.
 4. Mark the record as synchronized.
-

Updated Records

When editing an existing record:

1. Update SQLite.
 2. Queue the update.
 3. Synchronize with MySQL.
 4. Update synchronization status.
-

Deleted Records

Instead of immediately deleting:

1. Mark as deleted.
 2. Queue the deletion.
 3. Synchronize.
 4. Remove permanently only after successful synchronization (or retain as archived if using soft deletes).
-

Conflict Resolution

If two devices update the same record:

Compare LastModified Timestamp

↓

Newest Update Wins

↓

Create Audit Log

↓

Notify Administrator

Failed Synchronization

If synchronization fails:

- Keep the local record.
 - Retry automatically.
 - Notify the user that synchronization is pending.
 - Prevent duplicate uploads.
-

Synchronization Indicators

The application should clearly show synchronization status.

Example statuses:

- Synced
 - Pending Sync
 - Syncing
 - Failed
-

Chapter 18

Standard Error Messages

The application should display clear, user-friendly messages.

Situation	Message
Invalid Login	Invalid username or password.
User Disabled	This account has been disabled. Contact the administrator.
Duplicate Username	Username already exists.
Member Not Found	Member record could not be found.
Duplicate Phone Number	Phone number is already registered.

Duplicate QR Code	QR Code already exists.
Membership Expired	Membership has expired. Please renew before checking in.
Membership Paused	Membership is currently paused.
Duplicate Attendance	Attendance has already been recorded today.
Invalid Payment	Please enter a valid payment amount.
Payment Failed	Payment could not be completed.
Unauthorized Access	You do not have permission to perform this action.
Internet Required	No internet connection. Changes will sync automatically when connected.
Synchronization Failed	Unable to synchronize data. The system will retry automatically.
Unexpected Error	An unexpected error occurred. Please try again.

Chapter 19

Development Roadmap

Phase 1 – Foundation

- Create Solution
 - Configure GitHub Repository
 - Configure Branch Structure
 - Configure Database
 - Setup Entity Framework Core
 - Configure Dependency Injection
 - Configure Authentication
-

Phase 2 – Core System

Build in this order:

1. User Management
2. Member Management

3. Membership Plans
4. Memberships
5. Membership Pause

Each module must be fully completed before moving to the next.

Phase 3 – Daily Operations

Develop:

- Payments
 - Attendance
 - QR Attendance
 - Walk-In Visitors
 - Notifications
-

Phase 4 – Management Features

Develop:

- Reports
 - Settings
 - Audit Logs
-

Phase 5 – Offline & Synchronization

Implement:

- SQLite Database
 - Synchronization Queue
 - Automatic Synchronization
 - Conflict Resolution
 - Retry Logic
 - Synchronization Indicators
-

Phase 6 – Testing & Deployment

Complete:

- Unit Testing
- Integration Testing

- User Acceptance Testing
 - Bug Fixes
 - Performance Improvements
 - Final Documentation
 - Deployment
-

Definition of Done

A module is considered complete only when all of the following are finished:

- Database tables
 - Models
 - DTOs
 - Repository
 - Service
 - API endpoints
 - ViewModels
 - UI pages
 - Validation
 - Offline support (if applicable)
 - Testing
 - Documentation
-

Chapter 20

Future Improvements

The following features are intentionally excluded from Version 2.0.0 RC1 but may be considered in future releases.

Management

- Multi-Branch Gym Support
 - Trainer Management
 - Employee Management
 - Equipment Inventory
 - Payroll System
-

Member Features

- Member Mobile Portal
- Online Membership Renewal

- Online Appointment Booking
 - Push Notifications
 - Email Notifications
 - SMS Notifications
-

Payments

- Online Payment Gateway
 - Automatic Payment Verification
 - Subscription Auto-Renewal
-

Reporting

- Advanced Analytics Dashboard
 - Business Intelligence Reports
 - Revenue Forecasting
-

Other Enhancements

- Fitness Progress Tracking
 - Wearable Device Integration
 - Cloud Backup
 - Multi-Language Support
-

Conclusion

GymTrackPro is designed to modernize gym membership management by replacing manual processes with a centralized, secure, and easy-to-use digital system.

The project combines **.NET MAUI**, **ASP.NET Core Web API**, **Entity Framework Core**, **MySQL**, and **SQLite** to provide a responsive application that supports both online and offline operation.

By following the architecture, standards, and implementation plan outlined in this document, the development team can build a maintainable and scalable system while gaining practical experience with modern software engineering practices.

This specification serves as the primary reference for the development team and the Antigravity IDE Agent. Any significant architectural changes should be reviewed and documented before implementation to keep the project consistent and manageable.



Final Review (v2.0.0 RC1)

After seeing the complete document, I have **three final recommendations** before you start coding:

1. Add an ERD (Entity Relationship Diagram)

This is the only major artifact still missing. It will help both your team and the Antigravity IDE Agent understand the database relationships immediately.

2. Add a Version History page

Place this after the cover page.

Version	Date	Description
1.0.0	Initial Draft	Initial project specification
2.0.0 RC1	Current	Major rewrite with offline-first architecture, GitHub workflow, security improvements, user management, and expanded development standards

3. Add an Appendix

At the end of the document, include:

- Technology Stack (to be finalized later)
 - Project Glossary (JWT, API, Repository, MVVM, SQLite, etc.)
 - References (framework and documentation links)
 - Change Log
-

My Overall Assessment

If this were evaluated as a BSIT capstone specification, I'd rate it:

- **Completeness:** 9.8/10
- **Architecture:** 10/10
- **Maintainability:** 10/10
- **Student Scope:** 10/10
- **Developer Readiness:** 10/10

At this point, I would stop expanding the specification and move into implementation. As you build the project, you'll naturally discover small refinements, but the overall architecture and scope are now well-defined and stable.